



UNIVERSIDAD PRIVADA DEL VALLE
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROGRAMACIÓN WEB II

Manual Técnico After Dark

Paralelo: “A”

Estudiantes:

AMERICO OSWALDO RAMIREZ ROCHA

JULIO MATEO BUSTILLOS SERRANO

CRISTHIAN GIOVANNI PACSI PARI

Docente: Ing. Henry Miranda Ordóñez.

La Paz -Bolivia

Gestión I – 2026

Tabla de contenido

1. INTRODUCCIÓN.....	1
1.1 Propósito del Documento	1
1.2 Alcance del Sistema Web.....	1
1.3 Definiciones y Acrónimos	2
1.4 Referencias Técnicas	3
2. ARQUITECTURA DEL SISTEMA.....	3
2.1 Arquitectura Cliente-Servidor.....	3
2.2 Stack Tecnológico.....	4
2.3 Separación de Responsabilidades Frontend/Backend	5
3. DISEÑO DE LA BASE DE DATOS.....	6
3.1 Modelo Entidad-Relación.....	6
3.2 Diccionario de Datos	7
3.4 Procedimientos Almacenados.....	15
3.5 Vistas del Sistema.....	16
4. DESARROLLO DEL BACKEND (LARAVEL API).....	16
4.1 Estructura de Carpetas y Organización.....	16
4.2 Controladores API	17
4.3 Endpoints	18

5. DESARROLLO DEL FRONTEND (WEB VANILLA)	22
5.1 Estructura de Archivos y Organización	22
5.2 Módulos JavaScript y Responsabilidades.....	22
5.3 Gestión de Sesión y Estado Local	23
5.4 Estilos, Diseño y Validaciones en Cliente	24
6. SEGURIDAD Y AUTENTICACIÓN	25
6.1 Flujo de Autenticación con Laravel Sanctum	25
6.2 Control de Acceso Basado en Roles (RBAC)	25
6.3 Protección de Credenciales y Datos Sensibles	26
6.4 Prevención de CSRF y Configuración CORS	26
7. INSTALACIÓN Y CONFIGURACIÓN TÉCNICA.....	26
7.1 Requisitos de Servidor y Entorno.....	26
7.2 Configuración del Archivo .env.....	27
7.3 Comandos de Despliegue Local	28
8. CONCLUSIONES TÉCNICAS Y RECOMENDACIONES	28
8.1 Conclusiones del Desarrollo	28
8.2 Limitaciones Técnicas Identificadas.....	29

1. INTRODUCCIÓN

1.1 Propósito del Documento

Este Manual Técnico documenta la arquitectura, el diseño y los componentes internos del sistema web de gestión para el Karaoke-Bar After Dark. Está dirigido a desarrolladores y personal de soporte técnico que necesiten comprender cómo funciona el sistema por dentro, realizar mantenimiento o solucionar problemas técnicos.

El documento se centra exclusivamente en la plataforma web, describiendo las tecnologías utilizadas, la estructura de la base de datos, la lógica implementada mediante procedimientos almacenados y triggers, así como los mecanismos de seguridad y autenticación. Sirve como referencia para garantizar la continuidad operativa y facilitar futuras modificaciones al código fuente.

1.2 Alcance del Sistema Web

El sistema After Dark es una aplicación web con arquitectura cliente-servidor desacoplada. El frontend y el backend operan de forma independiente y se comunican únicamente a través de una API RESTful. El backend está construido con Laravel 9 sobre PHP, mientras que el frontend utiliza HTML5, CSS3 y JavaScript, sin frameworks externos ni herramientas de compilación.

Los módulos funcionales del sistema incluyen:

- **Gestión de Órdenes:** Creación, edición y cierre de cuentas, asignación de espacios y registro de productos consumidos.
- **Control de Inventario:** Registro de productos, ajustes manuales de stock y seguimiento automático de movimientos mediante triggers de base de datos.

- **Procesamiento de Pagos:** Registro de pagos parciales y totales con actualización automática del estado de la orden y cálculo de saldos pendientes.
- **Administración de Espacios:** Configuración de tipos de espacio, capacidades y tarifas por hora para rentas de cuartos privados.
- **Gestión de Usuarios:** Control de accesos basado en roles (Administrador y Empleado) mediante Pólices personalizadas
- **Reportes y Consultas:** Generación de informes de ventas, stock bajo y resumen de pagos mediante procedimientos almacenados.
- El sistema opera en entorno local mediante XAMPP. Toda la lógica de negocio reside en el backend Laravel; el frontend actúa solo como capa de presentación que consume los endpoints de la API.

1.3 Definiciones y Acrónimos

- **API REST:** Interfaz basada en HTTP que permite la comunicación entre frontend y backend mediante métodos estándar (GET, POST, PUT, DELETE) y formato JSON. Todos los endpoints están bajo el prefijo /api/.
- **Laravel Sanctum:** Paquete de autenticación de Laravel que genera tokens API para proteger rutas y verificar la identidad de usuarios mediante el header Authorization: Bearer {token}.
- **Trigger (Disparador):** Objeto de MySQL que se ejecuta automáticamente tras eventos INSERT, UPDATE o DELETE. En este sistema mantienen la integridad del inventario (trg_update_stock, trg_prevent_negative_stock) y calculan totales de órdenes (trg_update_order_total_after_insert, trg_auto_paid_status).
- **Procedimiento Almacenado:** Instrucciones SQL precompiladas en la base de datos que realizan operaciones complejas como reportes (sp_daily_sales_report) y análisis de stock (sp_low_stock_report), reduciendo la carga en el servidor de aplicaciones.
- **Fetch API:** Interfaz nativa de JavaScript para peticiones HTTP asíncronas al backend, manejando promesas y JSON sin librerías externas.

- LocalStorage: Almacenamiento persistente del navegador donde se guarda el token Sanctum y datos básicos del usuario durante la sesión.
- XAMPP: Entorno local con Apache, MySQL y PHP utilizado para ejecutar el sistema en desarrollo.
- Composer: Gestor de dependencias de PHP que instala las librerías requeridas por Laravel, incluyendo Sanctum.

1.4 Referencias Técnicas

- Laravel Framework Documentation v9.x: laravel.com/docs/9.x
- Laravel Sanctum Documentation: Documentación oficial del paquete de autenticación por tokens.
- MySQL 8.0 Reference Manual: Sintaxis de triggers, procedimientos almacenados y optimización de consultas.
- MDN Web Docs - Fetch API: Referencia técnica de la interfaz Fetch para comunicación asíncrona.
- W3C HTML5 Specification: Estándar del lenguaje de marcado usado en las vistas.
- PHP 8.x Manual: Documentación del intérprete PHP del servidor.

2. ARQUITECTURA DEL SISTEMA

2.1 Arquitectura Cliente-Servidor

El sistema After Dark funciona mediante una arquitectura cliente-servidor donde el frontend y el backend son dos componentes independientes que se comunican exclusivamente a través de una API RESTful. No existe renderizado del lado del servidor ni integración directa entre las capas; toda la interacción se realiza mediante peticiones HTTP estandarizadas que devuelven respuestas en formato JSON.

El frontend, desarrollado con tecnologías web nativas, actúa como un consumidor de datos responsable únicamente de la interfaz gráfica y la experiencia de usuario. El backend, construido con Laravel 9, centraliza toda la lógica de negocio, validaciones, seguridad y persistencia de datos. Esta separación permite que cada capa evolucione de forma independiente y facilita el mantenimiento al aislar responsabilidades técnicas específicas.

El flujo operativo sigue este patrón: el navegador ejecuta código JavaScript Vanilla que construye peticiones `fetch()` hacia los endpoints de la API, adjuntando el token de autenticación Sanctum en cada solicitud. El servidor Laravel procesa la petición a través de su pipeline de middleware, ejecuta la lógica correspondiente en el controlador API,

interactúa con la base de datos MySQL y devuelve una respuesta estructurada en JSON que el frontend interpreta para actualizar dinámicamente la interfaz sin recargar la página.

2.2 Stack Tecnológico

El sistema está construido sobre las siguientes tecnologías:

Backend:

- PHP 8.x: Intérprete del lado del servidor que ejecuta el framework Laravel y gestiona el procesamiento de peticiones HTTP.
- Laravel Framework 9.19: Framework PHP que proporciona la estructura de rutas, controladores API, modelos Eloquent, sistema de migraciones y gestión de dependencias mediante Composer.
- Laravel Sanctum 3.0: Paquete de autenticación que genera tokens API para proteger endpoints y gestionar sesiones de usuario mediante el header Authorization: Bearer {token}, eliminando la necesidad de cookies de sesión tradicionales.
- MySQL: Gestor de base de datos relacional que almacena toda la información transaccional y de catálogo del sistema.

Frontend Web:

- HTML5: Lenguaje de marcado estructural para las vistas estáticas del sistema (login.html, ordenes.html, inventario.html, etc.).
- CSS3: Hojas de estilo nativas organizadas por módulo funcional (login.css, ordenes.css, inventario.css) sin preprocesadores ni frameworks CSS externos.
- JavaScript ES6 (Vanilla): Lógica de cliente implementada en archivos JS independientes por funcionalidad (login.js, ordenes.js, inventario.js, usuarios.js, pagos.js, reportes.js), utilizando la Fetch API nativa para comunicación asíncrona y manejo de promesas.
- FontAwesome 6.5.1: Librería de íconos cargada vía CDN para elementos visuales de la interfaz.

Infraestructura Local:

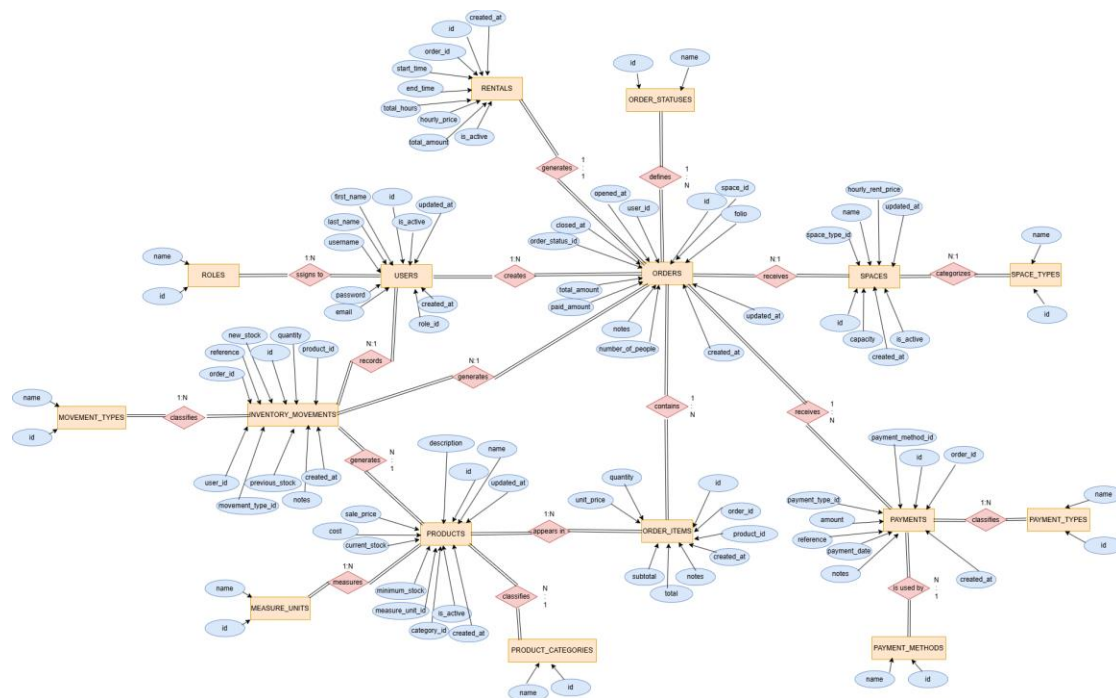
- XAMPP 8.2.12+: Entorno de desarrollo integrado que incluye Apache (servidor web), MySQL (base de datos) y PHP (intérprete). La versión específica depende de la máquina del desarrollador, pero debe ser compatible con PHP 8.x.
- Composer: Gestor de dependencias de PHP utilizado para instalar Laravel, Sanctum y todas las librerías requeridas por el proyecto.

2.3 Separación de Responsabilidades Frontend/Backend

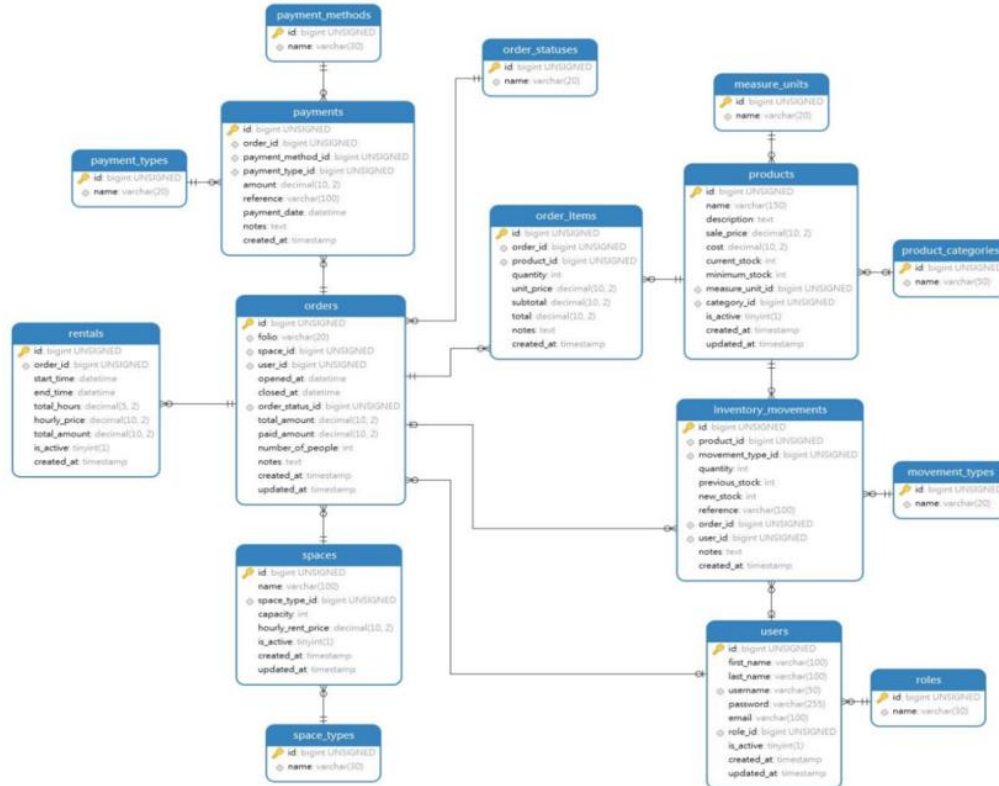
El sistema adopta un enfoque de separación estricta por capas comunicadas mediante API:

- **Capa de Presentación (Frontend):** Compuesta por archivos HTML estáticos servidos directamente desde el servidor web, hojas de CSS modulares y scripts JavaScript vanilla. Cada página HTML tiene asociado un archivo JS que encapsula toda la lógica de interacción, validación en cliente y consumo de API para esa funcionalidad específica. No existe templating del lado del servidor; el HTML se sirve como contenido estático y JavaScript se encarga de poblar dinámicamente los elementos del DOM con datos obtenidos de la API.
- **Capa de Lógica de Negocio (Backend API):** Implementada en Laravel bajo la estructura de controladores API ubicados en `app/Http/Controllers/Api/`. Cada controlador expone métodos que responden a verbos HTTP específicos (GET, POST, PUT, DELETE) y devuelven respuestas JSON estandarizadas. La validación de datos, reglas de negocio, autorización y persistencia se ejecutan exclusivamente en esta capa. Los middlewares `auth:sanctum` y `CheckRole` filtran las peticiones antes de que lleguen a los controladores, garantizando que solo usuarios autenticados y autorizados puedan acceder a recursos protegidos.
- **Capa de Datos:** Base de datos MySQL gestionada mediante migraciones Laravel y modelos Eloquent. Incluye lógica adicional implementada directamente en el motor de base de datos mediante triggers (`trg_update_stock`, `trg_auto_paid_status`) y procedimientos almacenados (`sp_daily_sales_report`, `sp_low_stock_report`) para operaciones críticas que requieren consistencia transaccional y rendimiento optimizado.

Esta arquitectura garantiza que el frontend sea agnóstico respecto a la implementación del backend: podría reemplazarse por cualquier otra tecnología (aplicación móvil, otro framework web) sin modificar la API, siempre que respete el contrato de endpoints y el formato de autenticación por tokens.



Modelo Físico:



El núcleo transaccional del sistema se centra en la tabla **orders**, que actúa como eje central conectando espacios (**spaces**), usuarios (**users**), estados de orden (**order_statuses**) e ítems de venta (**order_items**). Las tablas de catálogo (**products**, **payment_methods**, **space_types**, etc.) mantienen datos maestros independientes que son referenciados mediante claves foráneas desde las tablas operacionales. Esta separación permite actualizar catálogos sin afectar el historial transaccional y facilita consultas analíticas eficientes.

3.2 Diccionario de Datos

TABLA: users

Gestiona el acceso al sistema. Contiene nombres (**first_name**, **last_name**), credenciales de acceso (**username**, **password**), correo electrónico (**email**), rol asignado (**role_id**), estado activo/inactivo (**is_active**) y fechas de creación y actualización (**created_at**, **updated_at**). Las contraseñas se almacenan cifradas para garantizar la seguridad de la información.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	first_name	varchar(100)	No	Ninguna	—
3	last_name	varchar(100)	No	NULL	—
4	username	varchar(50)	No	Ninguna	Índice
5	password	varchar(255)	No	Ninguna	—
6	email	varchar(100)	Sí	NULL	—
7	role_id	int(11)	No	2	Índice
8	is_active	tinyint(1)	Sí	1	—
9	created_at	datetime	Sí	current_timestamp()	—
10	updated_at	datetime	Sí	current_timestamp()	ON UPDATE CURRENT_TIMESTAMP()

TABLA: space_types

Define los tipos de espacios físicos existentes en el establecimiento. Contiene el nombre del tipo de espacio (name), como Cuarto Privado, Mesa o Barra. Es utilizada por la tabla spaces para clasificar cada área de atención.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(30)	No	Ninguna	Índice

TABLA: spaces

Define los espacios físicos del establecimiento donde se atienden clientes. Incluye nombre (name), tipo de espacio (space_type_id), capacidad máxima de personas (capacity), tarifa por hora (hourly_rent_price), estado activo (is_active) y fechas de creación y actualización. Permite gestionar la disponibilidad y asignación de espacios durante la operación diaria.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(100)	No	Ninguna	—
3	space_type_id	int(11)	No	Ninguna	Índice
4	capacity	int(11)	Sí	NULL	—
5	hourly_rent_price	decimal(10,2)	Sí	0.00	—
6	is_active	tinyint(1)	Sí	1	—
7	created_at	datetime	Sí	current_timestamp()	—
8	updated_at	datetime	Sí	current_timestamp()	ON UPDATE CURRENT_TIMESTAMP()

TABLA: roles

Almacena los diferentes roles disponibles dentro del sistema. Contiene el nombre del rol (name), como Administrador, Cajero o Mesero. Permite controlar los permisos y funcionalidades disponibles para cada usuario según sus responsabilidades.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO INCREMENT (PK)
2	name	varchar(30)	No	Ninguna	Índice

TABLA: producto_categories

Catálogo de categorías utilizadas para clasificar los productos. Contiene el nombre de la categoría (name), permitiendo organizar el inventario en grupos como Cervezas, Whisky, Ron, Vodka, Cócteles u otras categorías definidas por el negocio.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(50)	No	Ninguna	Índice

TABLA: rentals

Gestiona el alquiler de espacios privados vinculados a órdenes. Contiene la orden asociada (order_id), hora de inicio (start_time), hora de finalización (end_time), cantidad total de horas utilizadas (total_hours), precio por hora aplicado (hourly_price), monto total generado (total_amount), estado activo (is_active) y fecha de creación (created_at). Permite calcular y facturar el uso de espacios alquilados independientemente del consumo de productos.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	order_id	int(11)	No	Ninguna	Índice
3	start_time	datetime	No	Ninguna	—
4	end_time	datetime	Sí	NULL	—
5	total_hours	decimal(5,2)	Sí	NULL	—
6	hourly_price	decimal(10,2)	No	Ninguna	—
7	total_amount	decimal(10,2)	Sí	NULL	—
8	is_active	tinyint(1)	Sí	1	—
9	created_at	datetime	Sí	current_timestamp()	—

TABLA: payment_types

Define los tipos de pago disponibles dentro del sistema. Contiene el nombre del tipo (name), normalmente Total o Parcial. Permite diferenciar entre pagos completos y abonos realizados a una orden.

#	Nombre	Tipo	Cotejamiento	Nulo	Predeterminado	Extra
1	id	int(11)	—	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(20)	utf8mb4_general_ci	No	Ninguna	Índice

TABLA: products

Catálogo maestro de productos disponibles para la venta. Contiene nombre (name), descripción (description), precio de venta (sale_price), costo de adquisición (cost), stock actual (current_stock), stock mínimo permitido (minimum_stock), unidad de medida (measure_unit_id), categoría (category_id), estado activo (is_active) y fechas de creación y actualización. Su stock es actualizado automáticamente mediante movimientos de inventario.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(150)	No	Ninguna	—
3	description	text	Sí	NULL	—
4	sale_price	decimal(10,2)	No	Ninguna	—
5	cost	decimal(10,2)	Sí	NULL	—
6	current_stock	int(11)	Sí	0	—
7	minimum_stock	int(11)	Sí	5	—
8	measure_unit_id	int(11)	No	Ninguna	Índice
9	category_id	int(11)	Sí	NULL	Índice
10	is_active	tinyint(1)	Sí	1	—
11	created_at	datetime	Sí	current_timestamp()	—
12	updated_at	datetime	Sí	current_timestamp()	ON UPDATE CURRENT_TIMESTAMP()

TABLA: payment_methods

Almacena los métodos de pago aceptados por el establecimiento. Contiene el nombre del método (name), como Efectivo, QR, Transferencia o Tarjeta. Es utilizada por la tabla payments para identificar la forma de pago utilizada por el cliente.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(30)	No	Ninguna	Índice

TABLA: payments

Registra todos los pagos efectuados sobre las órdenes. Incluye la orden asociada (order_id), método de pago (payment_method_id), tipo de pago (payment_type_id), monto pagado (amount), referencia externa (reference), fecha del pago (payment_date), observaciones (notes) y fecha de creación (created_at). Cada registro contribuye a actualizar automáticamente el saldo pendiente de la orden correspondiente.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	order_id	int(11)	No	Ninguna	Índice
3	payment_method_id	int(11)	No	Ninguna	Índice
4	payment_type_id	int(11)	No	Ninguna	Índice
5	amount	decimal(10,2)	No	Ninguna	—
6	reference	varchar(100)	Sí	NULL	—
7	payment_date	datetime	No	current_timestamp()	—
8	notes	text	Sí	NULL	—
9	created_at	datetime	Sí	current_timestamp()	—

TABLA: order_items

Registra el detalle de productos asociados a una orden. Incluye la orden relacionada (order_id), producto vendido (product_id), cantidad (quantity), precio unitario registrado al momento de la venta (unit_price), subtotal (subtotal), total calculado (total), observaciones (notes) y fecha de creación (created_at). Permite conservar el historial de precios aun cuando estos cambien posteriormente.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	order_id	int(11)	No	Ninguna	Índice
3	product_id	int(11)	No	Ninguna	Índice
4	quantity	int(11)	No	Ninguna	—
5	unit_price	decimal(10,2)	No	Ninguna	—
6	subtotal	decimal(10,2)	No	Ninguna	—
7	total	decimal(10,2)	No	Ninguna	—
8	notes	text	Sí	NULL	—
9	created_at	datetime	Sí	current_timestamp()	—

TABLA: order_statuese

Catálogo que almacena los estados posibles de una orden. Contiene el nombre del estado (name), como Abierta, Pagada, Cerrada o Cancelada. Permite controlar el flujo de trabajo y la situación actual de cada cuenta.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(20)	No	Ninguna	Índice

TABLA: orders

Almacena las órdenes o cuentas generadas dentro del establecimiento. Incluye folio único (folio), espacio asignado (space_id), usuario responsable (user_id), fecha de apertura (opened_at), fecha de cierre (closed_at), estado de la orden (order_status_id), monto total (total_amount), monto pagado (paid_amount), número de personas (number_of_people), observaciones (notes) y fechas de creación y actualización. Constituye la entidad central del proceso de ventas.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	folio	varchar(20)	No	Ninguna	Índice
3	space_id	int(11)	No	Ninguna	Índice
4	user_id	int(11)	Sí	NULL	Índice
5	opened_at	datetime	No	current_timestamp()	—
6	closed_at	datetime	Sí	NULL	—
7	order_status_id	int(11)	No	1	Índice
8	total_amount	decimal(10,2)	Sí	0.00	—
9	paid_amount	decimal(10,2)	Sí	0.00	—
10	number_of_people	int(11)	Sí	1	—
11	notes	text	Sí	NULL	—
12	created_at	datetime	Sí	current_timestamp()	—
13	updated_at	datetime	Sí	current_timestamp()	ON UPDATE CURRENT_TIMESTAMP()

TABLA: movement_types

Registra el detalle de productos asociados a una orden. Incluye la orden relacionada (order_id), producto vendido (product_id), cantidad (quantity), precio unitario registrado al momento de la venta (unit_price), subtotal (subtotal), total calculado (total), observaciones (notes) y fecha de creación (created_at). Permite conservar el historial de precios aun cuando estos cambien posteriormente.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(20)	No	Ninguna	Índice

TABLA: measure_units

Almacena las unidades de medida utilizadas para los productos del inventario. Contiene el nombre de la unidad (name), como Unidad, Botella, Caja, Litro o Mililitro. Facilita el control y administración de existencias.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	name	varchar(20)	No	Ninguna	Índice

TABLA: inventory_movements

Registra todos los movimientos realizados sobre el inventario. Incluye producto afectado (product_id), tipo de movimiento (movement_type_id), cantidad (quantity), stock anterior (previous_stock), stock resultante (new_stock), referencia relacionada (reference), orden asociada (order_id), usuario responsable (user_id), observaciones (notes) y fecha de registro (created_at). Garantiza la trazabilidad completa de las existencias.

#	Nombre	Tipo	Nulo	Predeterminado	Extra
1	id	int(11)	No	Ninguna	AUTO_INCREMENT (PK)
2	product_id	int(11)	No	Ninguna	Índice
3	movement_type_id	int(11)	No	Ninguna	Índice
4	quantity	int(11)	No	Ninguna	—
5	previous_stock	int(11)	Sí	NULL	—
6	new_stock	int(11)	Sí	NULL	—
7	reference	varchar(100)	Sí	NULL	—
8	order_id	int(11)	Sí	NULL	Índice
9	user_id	int(11)	Sí	NULL	Índice
10	notes	text	Sí	NULL	—
11	created_at	datetime	Sí	current_timestamp()	—

3.4 Procedimientos Almacenados

Los procedimientos almacenados encapsulan lógica compleja de reportes directamente en el motor de base de datos, reduciendo transferencia de datos y mejorando rendimiento:

sp_daily_sales_report(report_date DATE): Retorna el desglose de productos vendidos en una fecha específica, mostrando nombre del producto, cantidad total vendida y monto recaudado. Filtra únicamente órdenes cerradas y pagadas (`order_status_id = 2`) para reflejar ventas efectivas, no pendientes.

sp_low_stock_report(): Lista todos los productos activos cuyo stock actual es menor o igual al mínimo configurado. No requiere parámetros y se utiliza para alertas de reabastecimiento inmediato.

sp_orders_by_status(status_name VARCHAR(20)): Devuelve todas las órdenes que coinciden con un estado específico (ej. "Pendiente", "Pagado"), incluyendo folio, espacio asignado, montos y fecha de apertura. Permite filtrar vistas operativas según el flujo de trabajo del momento.

sp_top_products(top_count INT): Identifica los N productos más vendidos de todos los tiempos, ordenados por unidades despachadas. Útil para análisis de rotación y decisiones de compra. Solo considera órdenes pagadas para evitar distorsiones por cuentas abiertas.

sp_payment_summary(start_date DATE, end_date DATE): Genera resumen de ingresos por método de pago dentro de un rango de fechas, mostrando cantidad de operaciones y

monto total recaudado por cada medio (Efectivo, QR, Tarjeta). Base para arqueos de caja diarios y semanales.

3.5 Vistas del Sistema

Las vistas son consultas predefinidas almacenadas en la base de datos que simplifican accesos frecuentes a datos combinados:

- `vw_orders_summary`: Vista consolidada de órdenes con información de espacio, estado, montos y saldo pendiente calculado.
- `vw_sales_by_product`: Agregación histórica de ventas por producto con cantidades y montos acumulados.
- `vw_inventory_status`: Estado actual del inventario cruzando productos con sus últimos movimientos y niveles de alerta.
- `vw_payments_by_method`: Resumen de pagos agrupados por método con totales por período.
- `vw_unpaid_orders`: Filtro dinámico de órdenes con saldo pendiente mayor a cero, utilizado para seguimiento de cobranza.

Estas vistas son consumidas directamente por los endpoints de reportes del backend, evitando repetir lógica de joins y agregaciones en múltiples controladores.

4. DESARROLLO DEL BACKEND (LARAVEL API)

4.1 Estructura de Carpetas y Organización

El backend del sistema sigue la estructura estándar de Laravel 9, adaptada específicamente para funcionar como una API RESTful pura. La organización de archivos está diseñada para separar claramente la lógica de negocio de las rutas públicas y proteger el acceso mediante middleware.

- `app/Http/Controllers/Api/`: Contiene todos los controladores que responden a peticiones de la API. Cada entidad del sistema tiene su propio controlador (`OrderController`, `ProductController`, `AuthController`, etc.), agrupando métodos relacionados bajo un mismo archivo para facilitar el mantenimiento.
- `routes/api.php`: Archivo central donde se definen todas las rutas accesibles desde el frontend. Las rutas están protegidas por el middleware `auth:sanctum`, excepto `/login` que es pública. Aquí también se aplican middlewares específicos como `CheckRole` para restringir acciones administrativas.

- `app/Models/`: Alberga los modelos Eloquent que representan cada tabla de la base de datos. Estos modelos definen las relaciones entre entidades (ej. una Orden pertenece a un Espacio y tiene muchos Items) y facilitan la consulta de datos sin escribir SQL manual en los controladores.
- `database/migrations/`: Scripts versionados que crean y modifican la estructura de la base de datos. Incluye la migración especial `CreateProceduresAndTriggers` que inyecta la lógica de negocio directamente en MySQL.
- `app/Http/Middleware/`: Carpeta con filtros de seguridad personalizados, destacando `CheckRole.php` para validación de permisos y `Authenticate.php` integrado con Sanctum.

4.2 Controladores API

Los controladores actúan como intermediarios entre las peticiones HTTP y la base de datos. Su responsabilidad principal es recibir datos, validarlos, ejecutar la lógica necesaria y devolver una respuesta JSON estandarizada. No contienen lógica de presentación ni HTML.

- `AuthController`: Gestiona el inicio de sesión (login), cierre de sesión (logout) y obtención de datos del usuario actual (me). Genera tokens Sanctum tras validar credenciales contra la tabla `users`.
- `OrderController`: Maneja el ciclo de vida completo de las órdenes. Incluye métodos para listar, crear, actualizar y eliminar órdenes, además de funciones específicas como `addItem`, `updateItem` y `removeItem` para gestionar productos dentro de una cuenta activa.
- `ProductController` & `SpaceController`: Controladores CRUD estándar para catálogos. Permiten al administrador registrar nuevos productos o espacios, editar precios y capacidades, y desactivar elementos obsoletos.
- `PaymentController`: Procesa registros de pago. Al recibir una petición POST, crea el registro en `payments` y confía en el trigger `trg_auto_paid_status` para actualizar automáticamente el estado de la orden asociada.
- `ReportController`: Punto de entrada para consultas analíticas. En lugar de procesar datos en PHP, invoca directamente los procedimientos almacenados (`sp_daily_sales_report`, `sp_top_products`, etc.) y devuelve sus resultados formateados como JSON.
- `InventoryMovementController`: Registra ajustes manuales de stock y consulta el historial de movimientos. Valida que las salidas no superen el stock disponible antes de permitir la inserción.

4.3 Endpoints

Ordenes

Método	URL	Descripción	Token	Body de ejemplo
GET	/api/orders	Listar todas las órdenes (con espacio, usuario, estado, items)	Empleado/Admin	—
POST	/api/orders	Crear una nueva orden	Empleado/Admin	{ "space_id": 1, "number_of_people": 4, "notes": "Nota opcional" }
GET	/api/orders/{order}	Ver detalle de una orden (items, rentas, pagos)	Empleado/Admin	—
PUT	/api/orders/{order}	Actualizar datos generales (personas, notas) solo si está Pendiente	Empleado/Admin	{ "number_of_people": 5, "notes": "Cambio de mesa" }
DELETE	/api/orders/{order}	Cancelar la orden (solo Admin). Cambia estado a Parcial (ID 3)	Admin	—
POST	/api/orders/{order}/items	Agregar producto(s) a una orden pendiente	Empleado/Admin	{ "product_id": 1, "quantity": 2, "notes": "Sin hielo" }
PUT	/api/orders/{order}/items/{item}	Modificar cantidad de un item en orden pendiente	Empleado/Admin	{ "quantity": 5 }
DELETE	/api/orders/{order}/items/{item}	Eliminar un item de la orden pendiente (devuelve stock)	Empleado/Admin	—
POST	/api/orders/{order}/rentals	Iniciar renta de cuarto (solo si el espacio es tipo	Empleado/Admin	—

		Cuarto y orden pendiente)		
PUT	/api/rentals/{rental}/stop	Detener la renta activa (calcula horas y costo)	Empleado/Admin	—
GET	/api/orders/{order}/payments	Ver los pagos de una orden	Empleado/Admin	—
POST	/api/orders/{order}/payments	Registrar un pago (la orden se cierra automáticamente al cubrir total)	Empleado/Admin	{ "payment_method_id": 1, "payment_type_id": 1, "amount": 150, "reference": "Pago QR", "notes": "" }

Todos los endpoints requieren token Bearer en el encabezado Authorization

Las órdenes en estado Pagado o Parcial (diferente de order_status_id = 1) no permiten agregar/modificar/eliminar items, iniciar rentas ni pagos (excepto pagos mientras no esté completamente pagada)

Al agregar un producto, el stock se descuenta automáticamente; al eliminarlo, se reingresa

Usuarios

Método	URL	Descripción	Token	Body de ejemplo
GET	/api/users	Listar todos los usuarios (con rol)	Admin	—
POST	/api/users	Crear un nuevo usuario	Admin	{ "first_name": "Ana", "last_name": "López", "username": "analopez", "email": "ana@karaoke.com", "password": "secreto123", "role_id": 2 }
GET	/api/users/{user}	Ver un usuario específico	Admin	—
PUT	/api/users/{user}	Actualizar datos del usuario (nombre,	Admin	{ "first_name": "Ana María", "is_active": true }

		email, rol, etc.)		
DELETE	/api/users/{user}	Suspender al usuario (no se elimina físicamente)	Admin	—
PUT	/api/users/{user}/password	Cambiar la contraseña de un usuario	Admin	{ "password": "nuevaclave", "password_confirmation": "nuevaclave" }
PUT	/api/users/{user}/suspend	Suspender (o reactivar si está suspendido) al usuario	Admin	—

El campo role_id acepta 1 (Administrador) o 2 (Empleado)

La contraseña es obligatoria al crear un usuario; al editar, solo se envía si se desea cambiar (con confirmación)

Productos

Método	URL	Descripción	Token	Body de ejemplo
GET	/api/products	Listar productos activos (el campo cost solo es visible para Admin)	Empleado/Admin	—
GET	/api/products/{product}	Ver un producto	Empleado/Admin	—
POST	/api/products	Crear un nuevo producto	Admin	{ "name": "Cerveza Paceña", "description": "Botella 650ml", "sale_price": 15.00, "cost": 10.00, "current_stock": 100, "minimum_stock":

				20, "measure_unit_id": 1, "category_id": 1 }
PUT	/api/products/{product}	Editar un producto (precio, costo, stock, etc.)	Admin	{ "sale_price": 16.00, "cost": 11.00 }
DELETE	/api/products/{product}	Desactivar el producto (no se elimina)	Admin	—

Los empleados solo pueden leer; si intentan modificar, recibirán un error 403

Espacios

Método	URL	Descripción	Token	Body de ejemplo
GET	/api/spaces	Listar todos los espacios (con tipo)	Empleado/Admin	—
GET	/api/spaces/{space}	Ver un espacio	Empleado/Admin	—
POST	/api/spaces	Crear un nuevo espacio	Admin	{ "name": "Cuarto VIP", "space_type_id": 1, "capacity": 6, "hourly_rent_price": 70.00 }
PUT	/api/spaces/{space}	Editar un espacio (nombre, tipo, capacidad, precio hora)	Admin	{ "hourly_rent_price": 75.00, "capacity": 8 }
DELETE	/api/spaces/{space}	Desactivar el espacio	Admin	—

space_type_id corresponde a: 1 = Cuarto, 2 = Mesa, 3 = Barra.

Solo los cuartos generan renta por hora.

5. DESARROLLO DEL FRONTEND (WEB VANILLA)

5.1 Estructura de Archivos y Organización

El frontend del sistema After Dark está compuesto exclusivamente por archivos estáticos servidos directamente por el servidor web Apache (incluido en XAMPP). No existe un proceso de compilación, empaquetado ni transpilación; los archivos se ejecutan tal cual son escritos en el navegador. La organización sigue una separación estricta por tipo de recurso dentro de la carpeta assets/:

- assets/css/: Contiene hojas de estilo nativas organizadas por módulo funcional (login.css, ordenes.css, inventario.css, usuarios.css, pagos.css, reportes.css). Cada archivo estiliza únicamente su página HTML correspondiente, evitando dependencias cruzadas y facilitando la localización de reglas visuales.
- assets/js/: Alberga scripts JavaScript independientes por funcionalidad (login.js, ordenes.js, inventario.js, usuarios.js, pagos.js, reportes.js). Cada script encapsula toda la lógica de interacción, validación y consumo de API para su módulo específico, sin compartir estado global entre páginas.
- assets/imagenes/: Recursos gráficos estáticos utilizados en la interfaz (fondos, íconos decorativos).
- Archivos HTML raíz: Páginas principales (login.html, ordenes.html, inventario.html, usuarios.html, pagos.html, reportes.html) que actúan como contenedores estructurales. Cada HTML referencia explícitamente su CSS y JS asociados mediante etiquetas <link> y <script>.

Esta estructura modular permite que cada sección del sistema sea mantenible de forma aislada: si se requiere modificar la vista de inventario, solo es necesario editar inventario.html, inventario.css e inventario.js, sin riesgo de afectar otras funcionalidades.

5.2 Módulos JavaScript y Responsabilidades

Cada archivo JS representa un módulo autocontenido responsable de una funcionalidad específica del sistema:

- login.js: Gestiona el formulario de autenticación, envía credenciales al endpoint /api/login, almacena el token Sanctum y datos del usuario en localStorage, y redirige a la página principal tras login exitoso. Incluye manejo de errores de conexión y credenciales inválidas.
- ordenes.js: Módulo más complejo del sistema. Controla la creación de órdenes, selección de espacios, búsqueda y agregación de productos, gestión de rentas de

cuartos, procesamiento de pagos parciales/totales y cierre de cuentas. Mantiene estado local de la orden activa mediante variables globales temporales.

- **inventario.js:** Administra el catálogo de productos y espacios. Permite crear, editar y desactivar elementos, visualizar movimientos de stock históricos y realizar ajustes manuales de inventario. Implementa validaciones en tiempo real sobre formularios (solo letras, rangos numéricos, formatos de correo).
- **usuarios.js:** Exclusivo para administradores. Gestiona CRUD de usuarios del sistema, cambio de contraseñas, suspensión/reactivación de cuentas y visualización de detalles. Aplica filtros dinámicos por rol y estado sobre tablas de datos.
- **pagos.js:** Consulta historial de pagos, filtra por método y fecha, genera vistas previas de recibos y permite impresión directa. Calcula estadísticas de cobros diarios/mensuales en cliente.
- **reportes.js:** Consume endpoints de reportes administrativos, renderiza resultados en tablas dinámicas y aplica filtros de rango de fechas. No realiza cálculos complejos en cliente; delega toda la agregación a procedimientos almacenados del backend.

Todos los módulos comparten patrones comunes: función genérica `apiCall()` o `getHeaders()` para peticiones HTTP, manejo consistente de errores 401 (redirección a login) y 403 (mensaje de permiso denegado), y uso de `DOMContentLoaded` para inicializar eventos tras carga completa del DOM.

5.3 Gestión de Sesión y Estado Local

Dado que el sistema no utiliza cookies de sesión tradicionales, la persistencia del estado de autenticación depende completamente de `localStorage` del navegador:

- **Almacenamiento post-login:** Tras autenticación exitosa, `login.js` guarda dos claves: `token` (string del token Sanctum) y `usuario` (objeto JSON serializado con id, nombre, rol y email del usuario).
- **Verificación pre-carga:** Cada página incluye al inicio de su script una verificación `if (!token) { window.location.href = "login.html"; }` que impide el acceso a rutas protegidas sin token válido.
- **Lectura de perfil:** El rol y nombre del usuario se leen de `localStorage.getItem('usuario')` para personalizar la interfaz (mostrar/ocultar botones admin, saludar por nombre) sin necesidad de consultar nuevamente al servidor en cada navegación.

- Cierre de sesión: La función de logout llama a `/api/logout` para invalidar el token en servidor, luego ejecuta `localStorage.clear()` y redirige a login. Esto garantiza que tokens revocados no permanezcan accesibles en cliente.

Este enfoque tiene implicaciones de seguridad importantes: cualquier persona con acceso físico al navegador puede leer `localStorage`. Por ello, el token Sanctum configurado en el backend tiene expiración limitada y el middleware `auth:sanctum` valida cada petición independientemente del almacenamiento local.

5.4 Estilos, Diseño y Validaciones en Cliente

Estilizado: El diseño visual se implementa con CSS3 nativo, utilizando variables CSS para paleta de colores consistente (`#1a0b2e` fondo oscuro, `#00ff9d` acento verde neón, `#ff6b9d` alertas rosadas). No se emplearon frameworks CSS (Bootstrap, Tailwind) ni preprocesadores (Sass, Less). Los estilos están organizados por componente: bordes redondeados, sombras sutiles, tipografía legible y espaciado generoso para interfaces táctiles y de escritorio. FontAwesome 6.5.1 se carga vía CDN para íconos semánticos que mejoran la usabilidad sin aumentar peso de archivos locales.

Validaciones preventivas en cliente: Antes de enviar datos al servidor, los formularios implementan validación inmediata mediante JavaScript para mejorar experiencia de usuario:

- Filtrado de entrada en tiempo real: Campos como nombres de productos bloquean caracteres no alfabéticos mediante regex `/[^a-zA-ZáéíóúÁÉÍÓÚñÑ\s]/g` en evento `input`, limpiando automáticamente texto pegado o escrito. Nombres de espacios permiten además números para identificar "Cuarto 1", "Mesa VIP 2".
- Restricciones numéricas: Campos de cantidad y capacidad limitan valores máximos (ej. `capacidad ≤ 20 personas`) y previenen negativos interceptando teclas - y filtrando paste.
- Feedback visual inmediato: Inputs inválidos reciben clase CSS `.input-error` con borde rojo y mensaje descriptivo bajo el campo; inputs válidos obtienen `.input-valido` con borde verde. Esto ocurre en eventos `blur` e `input`, proporcionando retroalimentación antes del submit.
- Validación estructural al enviar: Funciones como `validarFormularioUsuarioCompleto()` recorren todos los campos configurados, verifican requeridos, longitudes mínimas/máximas, patrones regex y formatos de email, acumulando errores antes de permitir envío. Si hay fallos, se hace scroll suave al primer campo erróneo y se enfoca para corrección.

Estas validaciones son complementarias, no sustitutivas: el backend Laravel siempre revalida todos los datos recibidos mediante `$request->validate()`, ya que validación en cliente puede ser omitida fácilmente manipulando el navegador. Su propósito exclusivo es reducir viajes innecesarios al servidor y guiar al usuario durante la captura de datos.

6. SEGURIDAD Y AUTENTICACIÓN

6.1 Flujo de Autenticación con Laravel Sanctum

El sistema implementa un mecanismo de autenticación basado en tokens API mediante el paquete Laravel Sanctum. A diferencia de las sesiones tradicionales que dependen de cookies, este enfoque es ideal para aplicaciones desacopladas donde el frontend y el backend residen en dominios o puertos distintos.

El proceso inicia cuando el usuario ingresa sus credenciales (username y password) en el formulario de login. El frontend envía una petición POST `/api/login` al servidor. Si las credenciales son válidas, Laravel verifica el hash de la contraseña almacenado en la base de datos (generado con `bcrypt`) y genera un token de acceso único. Este token se devuelve en la respuesta JSON junto con los datos básicos del usuario.

El frontend recibe este token y lo almacena inmediatamente en el `localStorage` del navegador bajo la clave `token`. Para todas las peticiones subsiguientes a rutas protegidas, el script JavaScript recupera este valor y lo inyecta en el encabezado `HTTP Authorization: Bearer {token}`. El middleware `auth:sanctum` intercepta cada petición entrante, extrae el token, busca su registro en la tabla `personal_access_tokens` de la base de datos y valida que no haya expirado ni sido revocado. Solo si esta verificación es exitosa, la petición avanza hacia el controlador correspondiente.

6.2 Control de Acceso Basado en Roles (RBAC)

Una vez autenticado el usuario, el sistema debe determinar qué acciones tiene permitidas realizar. Esto se logra mediante el middleware personalizado `CheckRole`, ubicado en `app/Http/Middleware/CheckRole.php`.

Este middleware actúa como un filtro adicional después de la autenticación básica. Cuando una ruta requiere un rol específico (por ejemplo, `/reports/daily-sales` requiere ser Administrador), el middleware compara el nombre del rol asociado al usuario autenticado (`$request->user()->role->name`) con el rol exigido por la ruta. Si coinciden, la petición continúa; si no, se interrumpe inmediatamente devolviendo una respuesta HTTP 403 Forbidden con un mensaje JSON estandarizado: "No tienes permisos para realizar esta acción".

6.3 Protección de Credenciales y Datos Sensibles

La seguridad de las credenciales de usuario es fundamental. Las contraseñas nunca se almacenan en texto plano en la base de datos. Durante el registro o cambio de contraseña, Laravel aplica automáticamente el algoritmo de hashing bcrypt antes de persistir el dato. Al momento del login, Sanctum utiliza la función Hash::check() para comparar la contraseña ingresada contra el hash almacenado, sin necesidad de revertir el cifrado.

Además, el modelo User incluye por defecto la propiedad \$hidden = ['password'], lo que asegura que incluso si un endpoint devuelve accidentalmente el objeto completo del usuario, el campo de contraseña jamás será incluido en la respuesta JSON enviada al frontend.

Para datos transaccionales sensibles como costos de adquisición de productos, la lógica de filtrado se implementa directamente en el controlador ProductController. Si el usuario autenticado tiene rol "Empleado", el método index() elimina explícitamente el campo cost de la colección antes de devolverla. Así, el frontend nunca recibe información financiera interna, reduciendo la superficie de exposición de datos confidenciales.

6.4 Prevención de CSRF y Configuración CORS

Aunque Sanctum maneja autenticación por tokens, Laravel mantiene activas protecciones contra ataques Cross-Site Request Forgery (CSRF) para cualquier ruta que pueda recibir peticiones desde formularios HTML tradicionales. La configuración en config/cors.php define explícitamente los orígenes permitidos (allowed_origins) para aceptar peticiones desde el frontend local (http://localhost:8000 o similar). Esto previene que sitios maliciosos externos puedan enviar peticiones falsificadas aprovechando la sesión activa del usuario.

Asimismo, los headers de seguridad HTTP están configurados para rechazar peticiones que no incluyan el encabezado Accept: application/json, mitigando intentos de explotación mediante navegadores legacy o herramientas automatizadas que no respeten el contrato de la API REST.

7. INSTALACIÓN Y CONFIGURACIÓN TÉCNICA

7.1 Requisitos de Servidor y Entorno

Para ejecutar el sistema After Dark en un entorno local, es indispensable contar con una infraestructura que soporte las versiones mínimas requeridas por Laravel 9. Se

recomienda utilizar XAMPP como entorno integrado, ya que incluye todos los componentes necesarios en un solo paquete.

- Servidor Web: Apache 2.4.x (incluido en XAMPP).
- Intérprete PHP: Versión 8.0 o superior. Es crítico verificar que estén habilitadas las extensiones openssl, pdo_mysql, mbstring y tokenizer, ya que Laravel depende de ellas para funcionar correctamente.
- Base de Datos: MySQL 5.7+. El sistema utiliza características de claves foráneas y triggers que requieren un motor InnoDB configurado adecuadamente.
- Gestor de Dependencias: Composer instalado a nivel global en el sistema operativo. Sin esta herramienta no es posible descargar el framework ni sus librerías auxiliares.

La versión específica de XAMPP puede variar según la máquina del desarrollador, pero debe garantizar compatibilidad con PHP 8.x. Versiones anteriores a PHP 8.0 no son soportadas por Laravel 9.

7.2 Configuración del Archivo .env

El archivo .env actúa como el panel de control principal del sistema. Tras clonar o descargar el proyecto, este archivo no existe por defecto; se debe duplicar .env.example y renombrarlo como .env. Las variables críticas que deben configurarse manualmente son:

- APP_KEY: Clave criptográfica única generada automáticamente mediante el comando `php artisan key:generate`. Nunca debe compartirse ni modificarse manualmente después de su generación inicial, ya que cifra sesiones y datos sensibles.
- DB_CONNECTION: Debe mantenerse como `mysql`.
- DB_HOST / DB_PORT: Generalmente `127.0.0.1` y `3306` para entornos locales XAMPP.
- DB_DATABASE: Nombre de la base de datos creada en phpMyAdmin (ej. `karaoke_3`).
- DB_USERNAME / DB_PASSWORD: Credenciales de acceso a MySQL. En instalaciones típicas de XAMPP suele ser `root` sin contraseña, pero esto debe ajustarse según la configuración real del servidor.
- APP_URL: Dirección base donde corre el servidor de desarrollo (usualmente `http://localhost:8000`).

Cualquier error en estas variables impedirá que el sistema arranque o se conecte a la base de datos.

7.3 Comandos de Despliegue Local

Una vez configurado el entorno y el archivo .env, el despliegue sigue una secuencia estricta de comandos Artisan que preparan la aplicación:

1. Instalar dependencias: Ejecutar `composer install` en la raíz del proyecto. Este proceso lee el archivo `composer.lock` y descarga exactamente las versiones de librerías necesarias, incluyendo Laravel y Sanctum.
2. Generar clave de aplicación: Si no se hizo previamente, ejecutar `php artisan key:generate`. Esto escribe la clave cifrada en .env.
3. Preparar base de datos: Crear manualmente la base de datos vacía en phpMyAdmin con el nombre especificado en .env. Luego ejecutar `php artisan migrate --seed`. Este comando construye todas las tablas, relaciones, índices y además inyecta los triggers y procedimientos almacenados definidos en la migración `CreateProceduresAndTriggers`. Finalmente carga datos semilla como usuarios admin, categorías y productos base.
4. Iniciar servidor de desarrollo: Ejecutar `php artisan serve`. Esto levanta un servidor PHP integrado accesible en `http://localhost:8000`. La terminal donde se ejecuta este comando debe permanecer abierta durante toda la sesión de trabajo; cerrarla detiene inmediatamente el servicio.

8. CONCLUSIONES TÉCNICAS Y RECOMENDACIONES

8.1 Conclusiones del Desarrollo

El desarrollo del backend API con Laravel 9 y Sanctum demostró ser una arquitectura robusta para sistemas de gestión transaccional. La separación estricta entre frontend vanilla y backend permitió mantener un código limpio y modular, donde cada archivo JS tiene una responsabilidad única sin depender de frameworks pesados. Esto redujo significativamente la complejidad de despliegue local en XAMPP.

La implementación de lógica de negocio directamente en la base de datos mediante triggers (`trg_auto_paid_status`, `trg_update_stock`) y procedimientos almacenados resultó estratégica. Al delegar cálculos críticos como totales de órdenes y validación de stock al motor MySQL, se eliminó la posibilidad de inconsistencias por fallos en el código PHP y se mejoró el rendimiento en reportes complejos, ya que la agregación ocurre dentro del servidor de datos antes de transferir resultados.

El middleware personalizado `CheckRole` integrado con Sanctum proporcionó una capa de seguridad granular efectiva. A diferencia de soluciones genéricas, este enfoque permitió

restringir accesos a nivel de endpoint específico (ej. reportes y ajustes de inventario solo para administradores), garantizando que empleados autenticados no puedan acceder a funciones sensibles incluso si poseen un token válido.

8.2 Limitaciones Técnicas Identificadas

Ausencia de modo offline: El frontend depende 100% de conectividad activa al servidor local. Cualquier interrupción de red detiene completamente la operación, sin mecanismos de cola local o sincronización diferida.

Gestión manual de sesiones: Al usar localStorage para tokens, la expiración de sesión solo se detecta tras intentar una petición fallida (401). No existe renovación automática de tokens ni cierre de sesión por inactividad temporal.

Frontend no reactivo: La manipulación directa del DOM mediante JavaScript vanilla requiere recargas parciales manuales y gestión explícita de estado. Escalar a interfaces más complejas incrementaría exponencialmente la deuda técnica comparado con frameworks reactivos.

Respaldos no automatizados: El sistema carece de programación interna para backups de base de datos. La integridad de datos históricos depende exclusivamente de intervención manual administrativa.